

codestorywithMIK

CSwithMIK

codestorywithMIK

Recap : video - 12

Our AI agent now has improved a lot :

- → chat loop
- → Memory (Message history)
- → system prompt (Behaviour of Agent)
- → Tool Calling
- → Chained Tool Calling (Agent loop)
- → DEBUG Mode
- → LLM Response streaming

Before Building RAG, we must know :

AI Fundamentals List



How AI Actually Understands Meaning | What Are Embeddings? | AI Fundamentals Decoded | MIK

codestorywithMIK



Where Does AI Store Its Memories? | What is a Vector Database? | AI Fundamentals Decoded | MIK

codestorywithMIK



How to Give ChatGPT Your Private Data | What is RAG? | AI Fundamentals Decoded | codestorywithMIK

codestorywithMIK

Must watch...

Why do we even need RAG ???

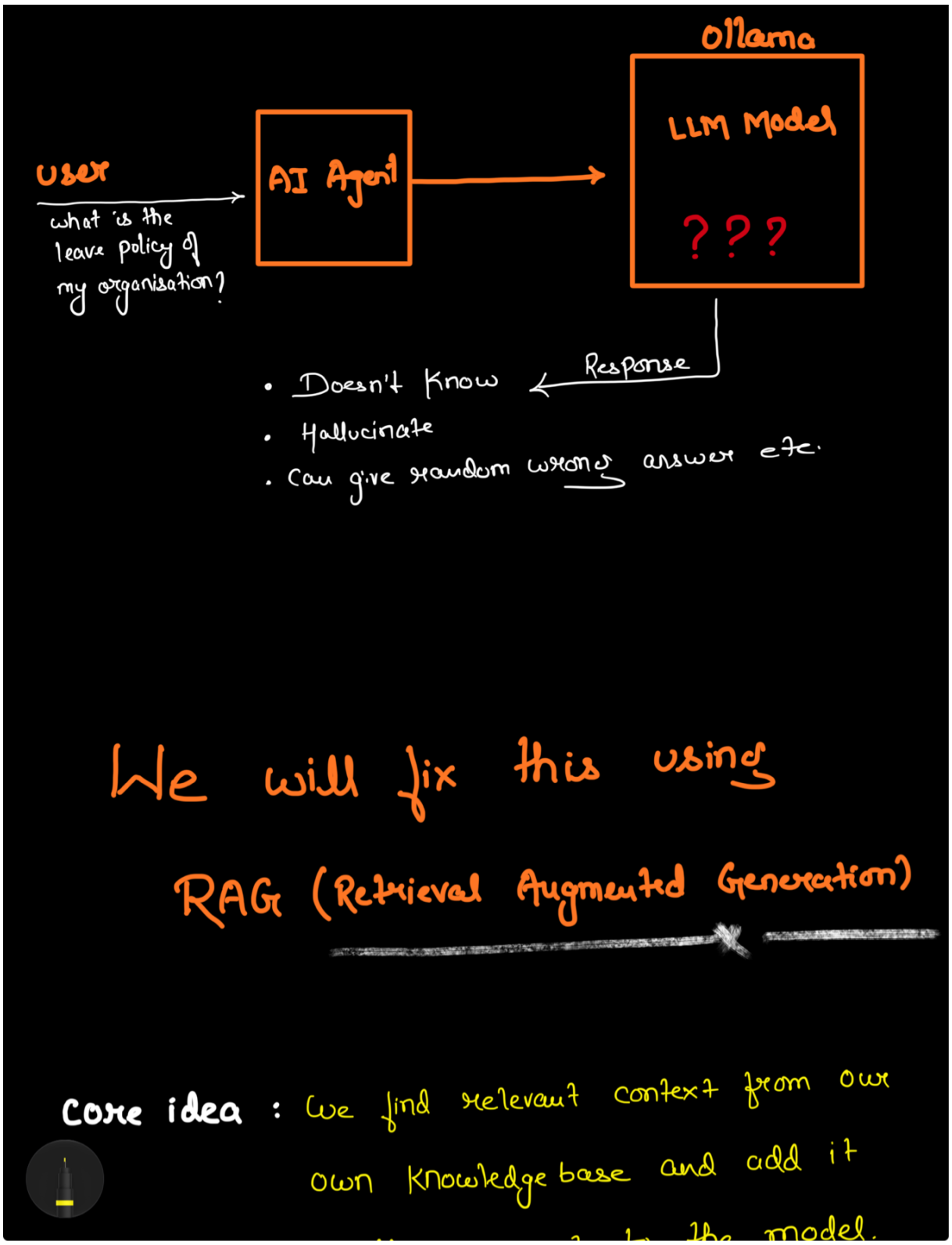
- Our Agent right now only knows things from two sources
  - Training done on AI Model
  - Tools we provided

- Suppose I want it to be an agent for

- Suppose, I want it to be an agent for my organisation.
- Every organisation has their own policies (Leave policies, medical policies etc.) -> Internal Information
- My Agent has no idea about those internal Information.

- Should we re-train our model with those knowledge
  - costly
  - slow

  and many more challenges.


Ollama
LLM Model
???
User
what is the leave policy of my organisation?
AI Agent
Response
• Doesn't Know
• Hallucinate
• Can give random wrong answer etc.
We will fix this using
RAG (Retrieval Augmented Generation)
Core idea : We find relevant context from our own knowledge base and add it

to the prompt to the model.
Model can frame response using
the knowledge we gave.

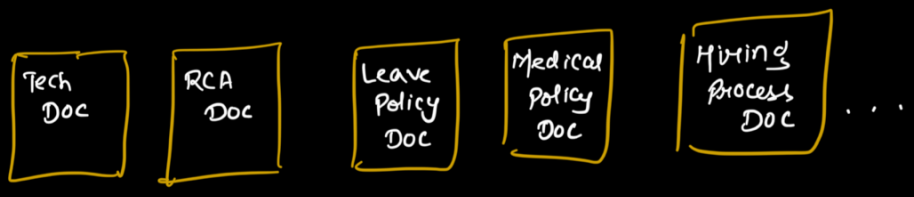
Relevant Context

- We need to search most relevant information related to use query.
- We can't do exact matching search of user query in Knowledge Base.
- We need a search that understands meaning – Semantic Search

User Query:

↳ eg "How many Sick Leaves are allowed in my Org. ?"

Knowledge Base (PDF/.txt etc.)



- Exact match won't always be possible ✗

User can ask different words like

'days off', 'vacation', 'holiday allowance' etc...

Leaves —Similar words→ day off, vacation, holiday etc.

That's why 'Embeddings' came to picture

text ⟶ token ⟶ tokenID ⟶ Embedding

Embedding = { . . . . . }

list of numbers/vector

* Similar meaning words give similar Embeddings
* Embedding captures semantic relationship.

car = [0.82, 0.10, -0.35, 0.61, 0.05, -0.22, 0.47, 0.09]....

automobile = [0.79, 0.14, -0.31, 0.58, 0.02, -0.25, 0.51, 0.06].....

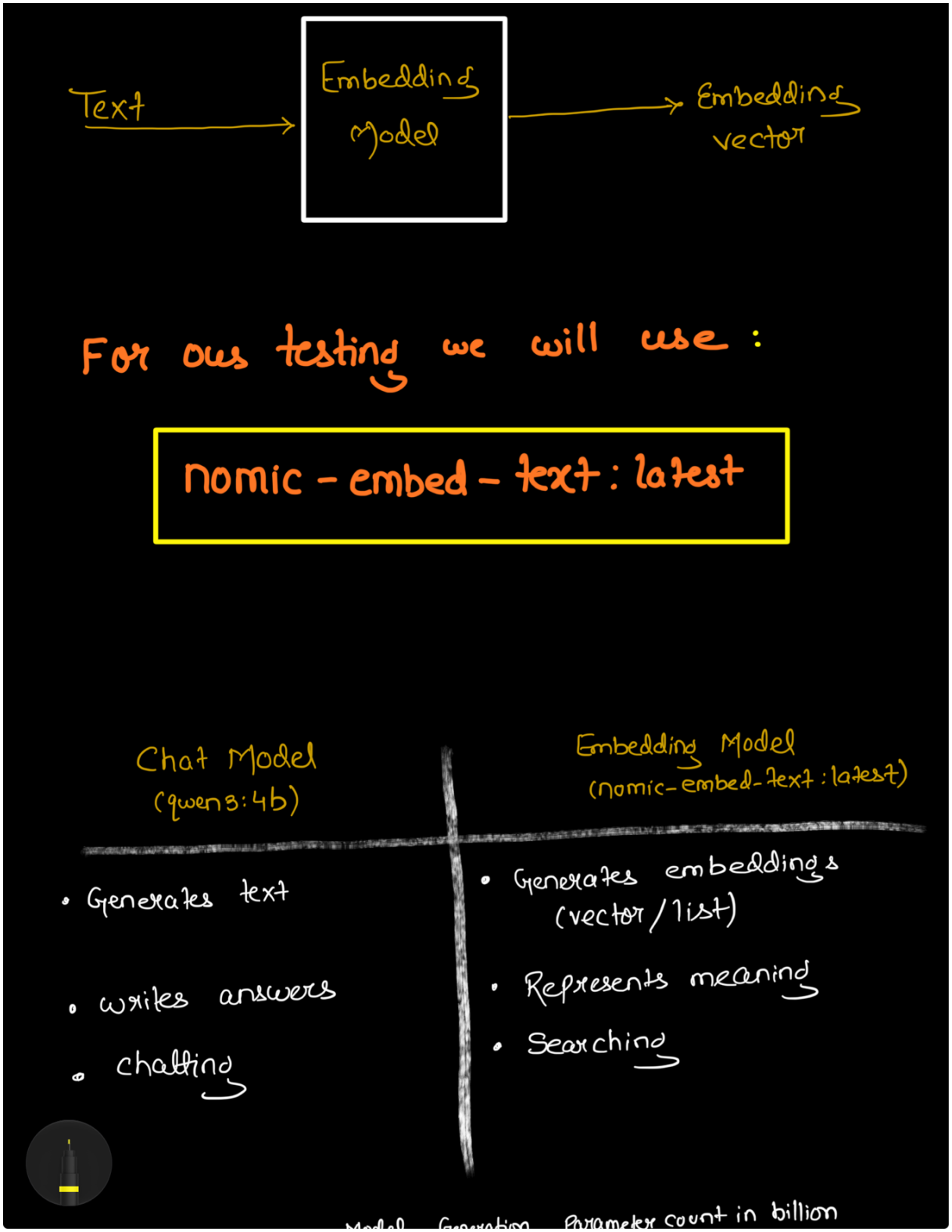
banana = [-0.12, 0.71, 0.44, -0.08, 0.66, 0.31, -0.20, 0.52].....

- It is usually a large vector (more dimensions)
- Embedding models learn these dimensions during training.
- Embedding models are designed in such a way that similar words get similar embeddings (via training)

* Good news is, we don't have to create embeddings of our own.

* KnowledgeBase → Chunks → Embeddings

The Embedding Model will do it


Text
Embedding Model
Embedding vector
For our testing we will use :
nomic-embed-text:latest
Chat Model (qwen3:4b)
Embedding Model (nomic-embed-text:latest)
• Generates text
• writes answers
• chatting
• Generates embeddings (vector / list)
• Represents meaning
• Searching
Model
Generation
Parameter count in billion

Chat Model = qwen 3 : 4b

(Model Family, Generation, Parameter count in billion)

Embedding Model =

[Provider/family] - [Task/Type] - [Architecture/Base] - [Dimension/size] - [version]

(dimension/size: optional (usually 768))

(nomic-embed-text : latest)

Let's download this embedding model in our ollama list

```
MIK ~
→ ollama list
NAME          ID              SIZE      MODIFIED
qwen3:4b      359d7dd4bcda    2.5 GB    3 weeks ago
MIK ~
→
```

How to download ?
(same way we did for qwen3:4b)

```
MIK ~
→ ollama list
NAME          ID              SIZE      MODIFIED
qwen3:4b      359d7dd4bcda    2.5 GB    3 weeks ago
MIK ~
→ ollama pull nomic-embed-text
pulling manifest
pulling 970aa74c0a90: 100% ▕████████████████████▏ 273 MB/274 MB  10.0 MB/s     0s
verifying sha256 digest
writing manifest
success
MIK ~
→
```

Check !

```
MIK ~
→ ollama list
NAME                          ID              SIZE      MODIFIED
nomic-embed-text:latest       0a109f422b47    274 MB    42 seconds ago
qwen3:4b                      359d7dd4bcda    2.5 GB    3 weeks ago
MIK ~
→
```

we got it

How will we test this model ?

Same way, we tested qwen3:4b

For chat models

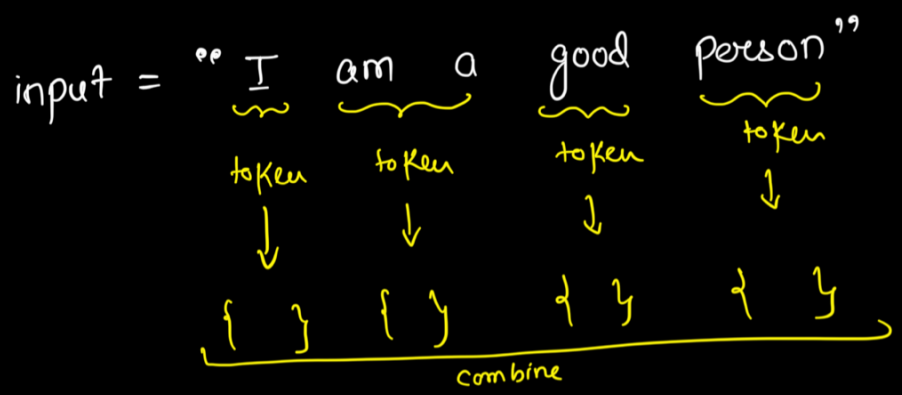
```
client.chat.completions.create(
    model=
    messages =
    tools =
    stream =
)
```

For Embedding models

```
client.embeddings.create(
    model="nomic-embed-text",
    input="Company leave policy..."
)
```

Let's test this out and check what we get in response...

```
client.embeddings.create(
    model= <model>
    input = <input>
)
```



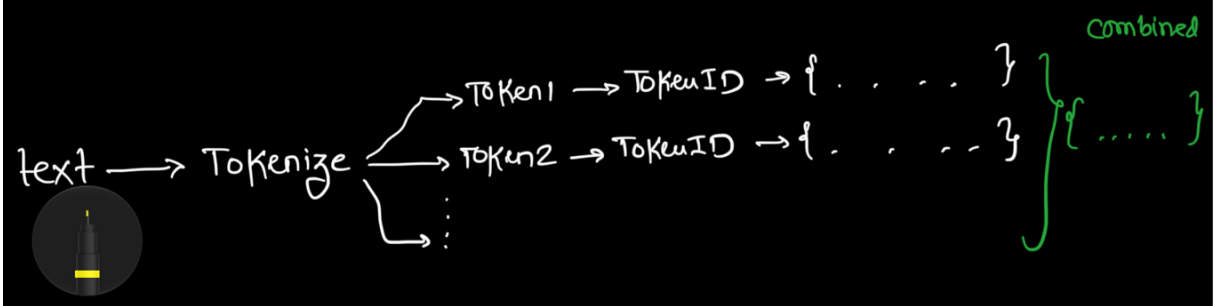
input = ["I am a good boy", "He is a coder"]

response.data[0].embedding → { . . . . }

response.data[1].embedding → { . . . . }

ie one embedding vector for one input"

Pooling ???





This process is called "Pooling"

How do we compare two vectors ??

text1 → embedding1 = { . . . . . }

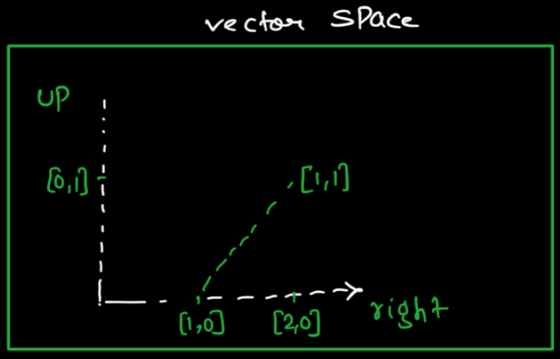
text2 → embedding2 = { . . . . . }

How to tell both texts are similar ?

Cosine Similarity

vector space

Cosine Similarity



Example: Embedding with only two dimension

[1,0] vector having length $= \sqrt{1^2+0^2} = 1$

[0,1] vector having length $= \sqrt{1^2+0^2} = 1$

[1,1] vector having length $= \sqrt{1^2+1^2} = \sqrt{2}$

[2,0] vector having length $= \sqrt{2^2+0^2} = 2$

* Cosine Similarity measures the angle between two vector arrows.

↳ Same direction → Similar

↳ Pointing apart → Not Similar

$$\text{Cosine Similarity} = \frac{(A \cdot B)}{|A|\,|B|}$$

(dot product)

$$\frac{[1,0] \cdot [2,0]}{1 * 2} = \frac{(1*2) + (0*0)}{1*2} = \frac{2}{2} = 1$$

↓ Similar

Close to 1 → Similar, same direction

Around 0 → unrelated ($\perp^r$)

Negative → opposite (rarely comes)

Note: These threshold values can vary for different models.

Python helps to do those calculations for you

numpy → (Numerical Python). year 2005

Python library → Helps to work with numbers in bulk, scientific calculation, massive calculation in huge data sets.

Cosine similarity :- ① Dot product
② length

```
import numpy
```

$\vec{A}$ = { }

$\vec{B}$ = { }

(•) numpy.dot(A, B)

(•) numpy.linalg.norm(A)

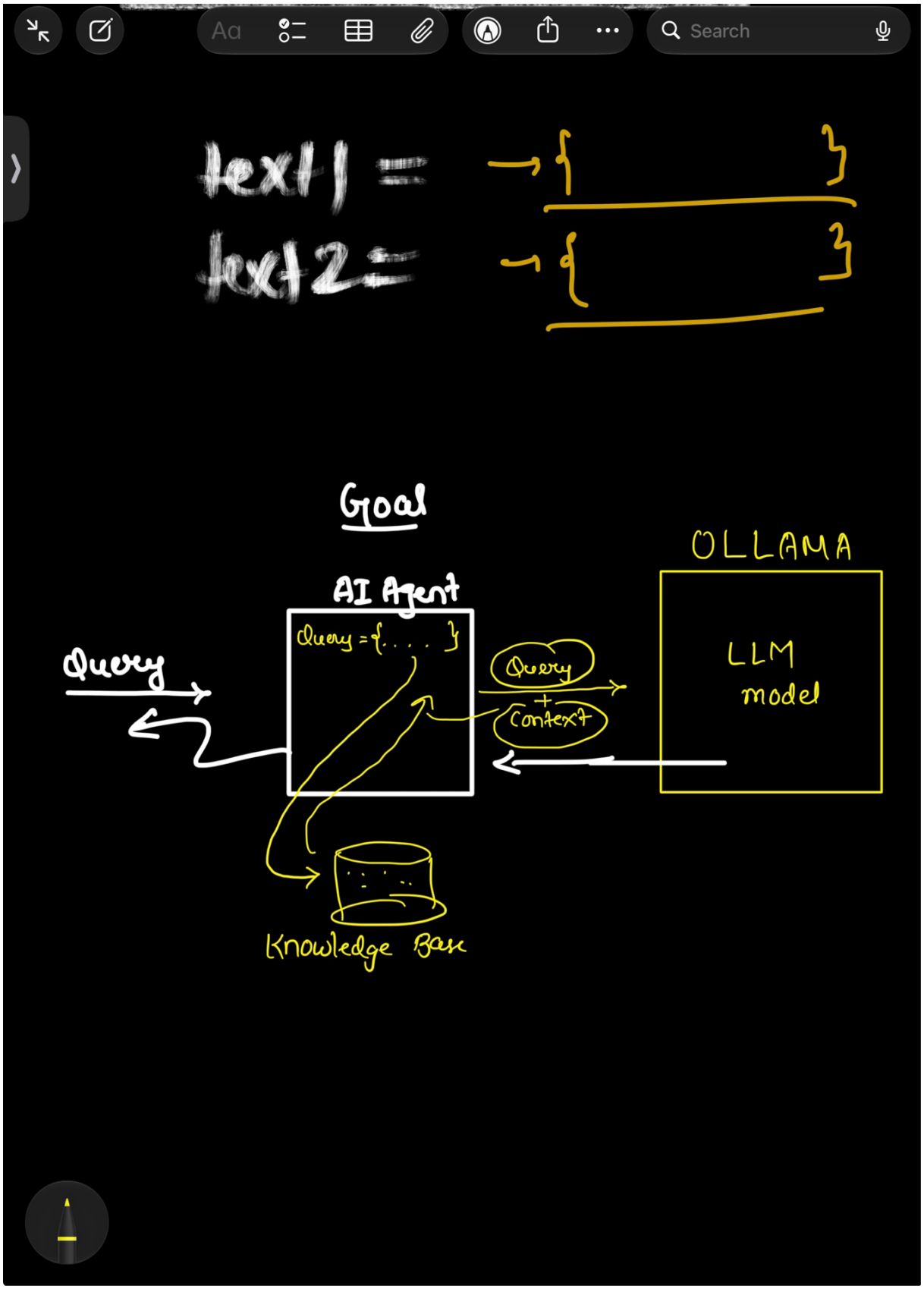
numpy.linalg.norm(B)

```
def cosine_similarity(A, B):

    dotProd = numpy.dot(A,B);
    LenA    = numpy.linalg.norm(A)
    lenB    = numpy.linalg.norm(B)
    return  dotProd/(lenA * lenB)
```

text1 = → { }

text2 = → { }


text1 =
text2 =
Goal
OLLAMA
AI Agent
Query = {. . . }
Query
Query
+
Context
LLM
model
Knowledge Base